

# Sztuczna inteligencja i inżynieria wiedzy

## Lista 5

Paweł Walkowiak

3 czerwca 2026

## 1 Cel listy

Celem listy jest zapoznanie z możliwościami modeli językowych na podstawie klasycznego zadania jakim jest klasyfikacja tekstu.

## 2 Wprowadzenie teoretyczne

### 2.1 Klasyfikacja tekstu

Klasyfikacja tekstu to jedno z fundamentalnych zadań Przetwarzania Języka Naturalnego (ang. *Natural Language Processing* – NLP). Polega ona na automatycznym przypisaniu etykiety (klasy)  $c \in C$  do nieustrukturyzowanego tekstu  $t$ , za pomocą klasyfikatora  $f$ . Z matematycznego punktu widzenia proces ten można przedstawić jako funkcję:  $f(t) \rightarrow c$ . W przypadku analizy wydźwięku, zbiór kategorii  $C$  obejmuje zazwyczaj wartości takie jak **plus** (pozytywny), **minus** (negatywny), **neutral** (neutralny) oraz **ambiguous** (mieszany). Współczesne podejście opiera się na reprezentacji tekstu w postaci gęstych wektorów liczb rzeczywistych, które kodują znaczenie semantyczne wypowiedzi.

### 2.2 Wielkie Modele Językowe (ang. LLM)

Wielkie Modele Językowe (ang. *Large Language Models* – LLM) to modele oparte na architekturze Transformer, posiadające miliardy parametrów i trenowane na gigantycznych zbiorach danych. Większość współczesnych modeli (np. GPT-4, Llama 3, Phi-2) to modele **generatywne typu Decoder-only**.

- **Zasada działania:** Modele te przewidują kolejny najbardziej prawdopodobny token (słowo lub fragment słowa) na podstawie dostarczonego kontekstu.
- **Zastosowanie w klasyfikacji:** LLMy nie wymagają trenowania do konkretnego zadania (tzw. *Zero-shot learning*). Poprzez odpowiednie sformułowanie instrukcji (*Prompt Engineering*), musimy na modelu wygenerowanie nazwy kategorii zamiast swobodnej odpowiedzi.

### 2.3 Modele typu Encoder-only (np. BERT)

Przed erą generatywnych LLM-ów, standardem były modele typu **Encoder-only**, których najpopularniejszym przedstawicielem jest BERT (*Bidirectional Encoder Representations from Transformers*).

- **Specjalizacja:** Modele te nie generują tekstu w sposób płynny, przekształcają tekst na wektor reprezentujący całe zdanie (często powiązany ze specjalnym tokenem [CLS]) co pozwala na klasyfikację z pomocą dodania odpowiedniej głowy klasyfikującej.
- **Wydażność:** Są znacznie mniejsze i szybsze niż modele generatywne, co czyni je idealnymi do zadań klasyfikacji masowej, gdzie liczy się szybkość i koszt obliczeniowy.

## 2.4 Środowisko pracy: Google Colab

W celu zapewnienia odpowiedniej mocy obliczeniowej niezbędnej do uruchomienia modeli językowych, zaleca się wykorzystanie środowiska **Google Colab**. Jest to bezpłatna usługa oparta na chmurze, która umożliwia pisanie i wykonywanie kodu Python w przeglądarce, oferując darmowy dostęp do akceleratorów sprzętowych **GPU** (np. NVIDIA T4).

Aby skorzystać z karty graficznej, co znacznie przyspieszy proces generowania tekstu (inferencję), należy po uruchomieniu notatnika zmienić typ środowiska wykonawczego:

1. Kliknij w menu **Runtime** (Środowisko wykonawcze).
2. Wybierz opcję **Change runtime type** (Zmień typ środowiska wykonawczego).
3. W sekcji **Hardware accelerator** (Akcelerator sprzętowy) wybierz **T4 GPU**.
4. Zatwierdź przyciskiem **Save**.

Wykorzystanie GPU pozwala na załadowanie modelu w formacie **bfloat16** lub **float16**, co redukuje zużycie pamięci VRAM i skraca czas oczekiwania na odpowiedź modelu.

## 3 Opis zbioru danych: PolEmo2.0-IN

W ramach niniejszego zadania zajmiemy się klasyfikacją wydźwięku tekstu (ang. *Sentiment Analysis*) na podstawie zbioru **allegro/klej-polemo2-in**. Jest to jeden z kluczowych komponentów benchmarku KLEJ (*Comprehensive Polish Language Evaluation*), służącego do oceny możliwości modeli językowych dla języka polskiego.

### 3.1 Charakterystyka zbioru

Zbiór **PolEmo2.0-IN** zawiera recenzje online z różnych dziedzin (np. medycyna, hotele). Zadanie polega na przypisaniu danej recenzji jednej z czterech kategorii emocjonalnych.

- **Źródło:** Recenzje użytkowników z serwisów internetowych.
- **Domena:** Recenzje z dziedzin medycyny i hotelarstwa (zbiór *in-domain*).
- **Liczba klas:** 4 kategorie wydźwięku:
  - **plus** – opinia jednoznacznie pozytywna,
  - **minus** – opinia jednoznacznie negatywna,
  - **neutral** – opinia opisowa, pozbawiona ładunku emocjonalnego,
  - **ambiguous** – opinia zawierająca zarówno elementy pozytywne, jak i negatywne (mieszana).

### 3.2 Struktura danych

Załadowany zbiór posiada następujące pola:

**sentence:** Treść recenzji w języku polskim (dane wejściowe dla modelu).

**target:** Identyfikator klasy (`__label__meta_minus_m`, `__label__meta_plus_m`, `__label__meta_zero`, `__label__meta_amb`).

### 3.3 Instrukcja ładowania zbioru

Do załadowania danych w środowisku Python należy wykorzystać bibliotekę **datasets** od Hugging Face:

```

from datasets import load_dataset

dataset = load_dataset("allegro/klej-polemo2-in", split="test")

print(f"Text:{dataset[0]['sentence']}")
print(f"Label:{dataset[0]['target']}")

```

Listing 1: Ładowanie zbioru PolEmo2.0

## 4 Przykłady ładowania modeli

### 4.1 Ładowanie modelu encoder-only:

```

from transformers import pipeline
import torch

encoder_model_name = "Voicelab/herbert-base-cased-sentiment"
sentiment_pipeline = pipeline("text-classification", model=encoder_model_name,
                              device=0 if torch.cuda.is_available() else -1)

sample_text = "Ala ma kota"
prediction = sentiment_pipeline(sample_text)
print(f"Prediction: {prediction}")

```

Listing 2: Ładowanie modelu za pomocą transformers.pipeline

### 4.2 Ładowanie LLM:

```

import torch
from transformers import pipeline, AutoModelForCausalLM, AutoTokenizer
from langchain_huggingface import HuggingFacePipeline
from langchain_core.prompts import PromptTemplate

llm_model_name = "Qwen/Qwen2.5-1.5B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(llm_model_name)
model = AutoModelForCausalLM.from_pretrained(
    llm_model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

hf_pipeline = pipeline("text-generation", model=model, tokenizer=tokenizer,
                      temperature=0.1, do_sample=True, pad_token_id=tokenizer.eos_token_id)
llm = HuggingFacePipeline(pipeline=hf_pipeline)

basic_template = """Classify the text sentiment into one of three classes:
    positive, negative, neutral.
Text: {text}
Class: """

prompt = PromptTemplate.from_template(basic_template)
llm_chain = prompt | llm

result = llm_chain.invoke({"text": sample_text})
print(result)

```

Listing 3: Ładowanie LLM

## 5 Zadanie

Twoim zadaniem jest opracowanie porównanie klasyfikacji na zadanym zbiorze w podejściach z użyciem modeli encode i decoder. W każdym z przypadków należy zaraportować wyniki za pomocą standardowych miar jak Accuracy, F1 oraz zwrócić uwagę na jakość klasyfikacji dla poszczególnych klas. Należy zwrócić szczególną uwagę na mapowanie etykiet tekstowych (wygenerowanych przez model) na identyfikatory numeryczne zgodne ze strukturą zbioru.

### Punktacja:

1. Załadowanie zbioru danych PolEmo2.0-IN, sprawdzenie podstawowych statystyk zbioru (liczność próbek w podzbiorach, zrównoważenie klas, jak długie są teksty). UWAGA: do celów zadania należy użyć tylko przykładów ze zbioru testowego bez klasy ambiguous. (10 punktów)
2. Przeprowadzenie klasyfikacji z użyciem pretrenowanego (ang. pretrained) modelu encoder-only. Przykładowy model polecany na początek: `Voicelab/herbert-base-cased-sentiment`. (20 punktów)
3. Eksploracja parametrów w celu poprawy/porównania wyników klasyfikacji (temperatura, inny model klasyfikujący). Dostępne modele można wyszukać na platformie [https://huggingface.co/models?pipeline\\_tag=text-classification](https://huggingface.co/models?pipeline_tag=text-classification) (20 punktów)
4. Przeprowadzenie klasyfikacji za pomocą modelu decoder-only LLM (należy dobrać rozmiar modelu do dostępnych zasobów obliczeniowych). Model polecany na początek: `Qwen/Qwen2.5-1.5B-Instruct` (20 punktów)
5. Eksploracja parametrów w celu poprawy wyników klasyfikacji (prompt, temperatura, kwantyzacja, parsowanie wyników do np. za pomocą `langchain_core.output_parsers.JsonOutputParser`). (30 punktów)

Eksploracja parametrów powinna sprawdzać co najmniej dwa aspekty, przykładowo: jak temperatura generacji wpływa na wyniki i model bazowy wpływa na wyniki klasyfikacji. Proszę nie przejmować się wykorzystywanie mniejszych modeli (typu 1.5B/3B) zamiast większych, w zadaniu chodzi o zbudowanie intuicji w użyciu LLMów, która przyda się w przyszłych projektach.

Do zadania przygotuj raport zawierający krótki opis wszystkich wykonywanych kroków oraz rezultatów zadania wraz z interpretacją. W raporcie wskaż wykorzystane materiały źródłowe oraz krótko opisz biblioteki wykorzystane przy implementacji. Raport wyślij prowadzącemu przynajmniej na 24 godziny przed oddaniem listy.

## 6 Dodatek - przydatne biblioteki dla Pythona

- `import torch`
- `from transformers import pipeline`
- `from datasets import load_dataset`
- `from langchain_huggingface import HuggingFacePipeline`
- `from langchain_core.prompts import PromptTemplate`
- `from langchain_core.output_parsers import JsonOutputParser`
- `from pydantic import BaseModel, Field`